BPF-DAG: Byte-Packet-Flow Features Fusion via Dynamic Attributed Graph for Reliable Encrypted Traffic Classification

Yunxiao Shi, *Student Member, IEEE*, Gaolei Li, *Member, IEEE*, Jun Wu, *Senior Member, IEEE*, Jianhua Li, *Senior Member, IEEE*, and He Fang, *Member, IEEE*

***Abstract*—Reliable encrypted traffic classification is crucial for fine-grained and efficient network security management, enabling accurate user behavior recognition and cybercrime forensics. While AI-based methods can automatically extract subtle features from traffic data, existing approaches often fail to effectively capture and integrate features across different levels of traffic granularity, namely the byte, packet and flow levels. Current graph-based methods heavily rely on manual feature engineering to construct global IP-based graphs, overlooking critical packet-level temporal features and byte-level raw information. Focusing on only one or two levels of traffic granularity is unreliable and insufficient, ultimately compromising model accuracy and robustness. To address these limitations, we propose BPF-DAG, a byte-packet-flow feature fusion framework based on dynamic attributed graphs, for reliable encrypted traffic classification. To the best of our knowledge, this is the first method that integrates temporal packet relations into flow interaction patterns while directly leveraging raw byte-level data. Specifically, we introduce a multi-granularity feature fusion strategy that dynamically updates an IP-based graph by iteratively assigning edge attributes derived from evolving flow representations. During the joint training of the Transformer and the graph neural network, temporal representations are learned from raw packet sequences and reflected in edge attributes dynamically for further message aggregation. Experiments on the ISCX VPN-nonVPN, Tor-nonTor, MIRAGE-2019 and MIRAGE-2024 datasets show that BPF-DAG outperforms recent state-of-the-art methods in terms of classification performance.**

***Index Terms*—Encrypted traffic classification, graph neural network, features fusion, transformer, dynamic attributed graph.**

I. Introduction

NOWADAYS, various encryption technologies are extensively used to encrypt network traffic, protecting users' privacy and ensuring anonymity [1]. Since 2019, the proportion of encrypted web traffic has exceeded 90 percent [2]. Meanwhile, the widespread adoption of privacy-enhanced technologies like virtual private network (VPN) and The Onion Router (Tor) [3] is paralleled by a sharp increase in malwares and cyberattacks [4], [5], [6]. Encrypted technologies have been abused by cybercriminals to conceal malicious behaviors, making it difficult to trace traffic sources and posing significant challenges to cybersecurity. Encrypted traffic classification is a fundamental task in the field of cybersecurity, serving as the basis for several tasks such as application identification [7], website fingerprinting [8], and malicious intrusion detection [9]. Highly accurate encrypted traffic classification enables robust user behavior recognition and cyber-crime forensics.

The adoption of encryption protocols significantly curtailed the sufficient exploitation of traffic information. Traditional rule-based methods, such as deep packet inspection (DPI) [10], [11], are no longer effective when confronted with encrypted payloads. In recent years, a range of ML-based methods have demonstrated strong performance in encrypted traffic classification [12], [13], [14], [15], [16]. Machine learning (ML)-based methods typically involve two steps: first, extracting hand-crafted statistical features, followed by utilizing traditional machine learning models, such as support vector machines (SVM) and random forests. But the manual feature extraction process is time-consuming and heavily dependent on expert knowledge. Additionally, the granularity of extracted traffic information is often too limited to achieve the reliable encrypted traffic classification. With the rapid advancement of AI [17], [18], [19], [20], deep learning (DL)-based methods have been proposed and have shown excellent performance, as they can automatically extract latent features from raw traffic data [21], [22], [23], [24], [25], [26], [27]. DL-based methods typically transform traffic data into image-like or natural language formats, allowing for the extensive application of computer vision (CV) and natural language processing (NLP) technologies. However, these methods fail to exploit topological patterns in host-to-host communications, i.e., the flow-level traffic information. Fully leveraging the interaction patterns among hosts across a network can further enhance the precision of traffic detection. Recently, graph neural networks (GNNs) [28], [29], [30], [31] have attracted extensive attention for their powerful ability to represent graphs, leading to

Received 15 December 2024; revised 7 June 2025 and 15 October 2025; accepted 30 November 2025. Date of publication 11 December 2025; date of current version 26 December 2025. This work was supported by the National Natural Science Foundation of China under Grant 62202303, Grant U21B2019, Grant 62471301, and Grant 62572314. The associate editor coordinating the review of this article and approving it for publication was Dr. Meng Li. *(Corresponding author: Gaolei Li.)*

Yunxiao Shi, Gaolei Li, Jun Wu, and Jianhua Li are with the School of Electronic Information and Electrical Engineering, Shanghai Jiao Tong University, Shanghai 200240, China, and also with Shanghai Key Laboratory of Integrated Administration Technologies for Information Security, Shanghai 200240, China (e-mail: shiyunxiao@sjtu.edu.cn; gaolei_li@sjtu.edu.cn; junwuhn@sjtu.edu.cn; lijh888@sjtu.edu.cn).

He Fang is with the College of Computer and Cyber Security, Fujian Normal University, Fuzhou 350117, China (e-mail: fanghe@fjnu.edu.cn).

Digital Object Identifier 10.1109/TIFS.2025.3643127

1556-6021 © 2025 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on September 18,2026 at 06:22:17 UTC from IEEE Xplore. Restrictions apply.

breakthroughs in fields such as chemistry, physical systems and social networks, among others. GNNs are capable of capturing latent topological patterns via message passing between nodes or edges of a graph. GNN-based methods for encrypted traffic classification can generally be categorized based on their graph construction strategies. One common method involves constructing a packet-level graph, where the packet length sequence of each bidirectional flow is used to model packets as graph nodes. This effectively transforms the traffic classification task into a graph classification problem. However, this approach analyzes each flow independently, lacking a global view of the network connectivity. Another common method attempts to capture traffic interaction patterns by constructing a flow-level global communication graph, where IP addresses are modeled as nodes and traffic flows between hosts are represented as edges. Yet, these approaches rely heavily on feature engineering to initialize graph attributes, without sufficiently leveraging byte-level traffic.

To achieve more reliable and accurate encrypted traffic classification, it is imperative to comprehensively exploit multi-granular traffic information to capture discriminative features. In this paper, we propose BPF-DAG, a byte-packet-flow feature fusion-based method via dynamic attributed graphs for reliable encrypted traffic classification. This is the first method that integrates temporal packet relations into flow interaction patterns while directly extracting byte-level features from raw traffic. We introduce a novel multi-granularity feature fusion strategy that comprehensively integrates byte-level, packet-level, and flow-level traffic features, thereby improving the robustness and effectiveness of the classification.

Specifically, BPF-DAG is composed of three core modules: (1) raw information extraction, (2) flow representation learning, and (3) multi-granularity feature fusion. The first module separately extracts topological and temporal information from raw traffic. The topological information is utilized to construct a global IP-based directed graph that represents flow-level network connectivity. The temporal information refers to the sequences of raw byte packets within each flow, encompassing both packet-level temporal dynamics and byte-level intrinsic features. In the second module, temporal and topological representations are learned in parallel from the extracted information. Specifically, the Transformer model is employed to capture sequential dependencies in the raw byte-based packet sequences, and the resulting representations are used to initialize the edge attributes in the graph. Concurrently, we propose a novel graph neural network, Directed Edge Sample and Aggregate (DiESAGE), to learn directed edge embeddings by iteratively propagating messages across IP nodes. The output of DiESAGE serves as the flow-level topological representation. In the third module, the temporal and topological representations are fused via linear interpolation to produce the final representation for each flow. During training, temporal flow representations are recomputed after each epoch using the updated Transformer, and the corresponding graph edge attributes are dynamically reassigned. To mitigate memory limitations during joint training of the Transformer and DiESAGE, we adopt a memory bank mechanism that tracks all temporal representations and updates a subset corresponding to the sampled mini-batch.

In summary, the key contributions of our work are outlined as follows:

- We propose a byte-packet-flow feature fusion-based reliable encrypted traffic classification method via dynamic attributed graph, called BPF-DAG. BPF-DAG captures packet-level temporal characteristics within each flow and flow-level topological patterns within a global IP-based graph. To the best of our knowledge, this is the first method that integrates packet relations into flow interaction patterns while directly extracting byte-level traffic features, for reliable encrypted traffic classification.
- Our approach introduces a multi-granularity features fusion strategy based on dynamic attributed graph, where graph attributes are dynamically assigned with evolving flow representations during each iteration. Temporal representations of flows are computed using an optimized Transformer model after each training epoch, with which the corresponding directed edge attributes are then updated. To reduce the memory overhead, we leverage a technique called Memory Bank to track all temporal representations to decouple the training batch size from the total number of edges.
- To evaluate the classification performance of the proposed BPF-DAG, we compare it with several baselines on six public encrypted traffic datasets, including ISCX-VPN, ISCX-nonVPN, ISCX-Tor, ISCX-nonTor, MIRAGE-2019 and MIRAGE-2024. Experiment results show that BPF-DAG not only outperforms existing SOTA methods by 1.4%, 1.2%, 2.3%, 0.2%, 7.7%, 3.1% in terms of accuracy, but also exhibits relatively lower complexity.

The structure of the paper is as follows: Section II reviews related work, covering ML-based, DL-based, and GNN-based methods, and highlights the differences between our work and existing approaches. Section III introduces the preliminaries. In Section IV, we describe the designs of our method, including the introduction of overall pipeline and the detailed functions of three sub-modules. In Section V, we experimentally evaluate the performance of our method, including classification performance, few-shot analysis and complexity analysis and compare our results with SOTA methods. Finally, Section VI concludes the paper and discusses future directions.

II. Related Work

A. ML-Based Methods

ML-based methods [12], [13], [14], [15], [16], [40] typically consist of two stages: feature engineering and model training. Feature engineering refers to manually extracting statistical features, such as mean packet length, duration, etc. These features are then used as input for ML models such as SVM and Naive Bayes to obtain final classification results. Appscanner [12] takes statistical features derived from packet length sequences as input for training Random Forest classifier. Anderson et al. [41] extract 198 TLS features, including flow metadata, byte distribution, packet lengths, and then apply

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on September 18,2026 at 06:22:17 UTC from IEEE Xplore. Restrictions apply.

a logistic regression classifier to classify encrypted traffic. Shi et al. [32] propose a feature optimization approach that efficiently reduces redundant statistical features and mitigates the negative effects of class imbalance. NB-SVM [15] utilizes a naive Bayes feature embedding method to transforms the original features into high-quality data, which is then classified using an SVM. However, ML-based methods depend heavily on expert-designed features and fail to fully exploit raw byte traffic. In contrast, our method does not rely on feature engineering and directly utilizes byte-level traffic data for classification.

B. DL-Based Methods

DL-based methods are capable of automatically extracting features from raw traffic data, enabling a unified end-to-end framework. Wang et al. were the first to apply representation learning to traffic classification by transforming raw traffic into grayscale images and utilizing convolutional neural networks (CNNs), including both 1D-CNN [21] and 2D-CNN [42], to perform image classification. Building on this, some works [24], [25], [43] have combined CNNs with RNNs to capture temporal characteristics from time-related raw packet sequences. Other methods, such as DF [22] and FS-Net [23], employ DL models to extract sequential features from packet length sequences. XAI [44] investigates trustworthiness and interpretability via explainable AI-based techniques to understand, interpret and improve the behavior of multimodal DL traffic classifiers. Additionally, the pre-training strategy have been applied to encrypted traffic classification by leveraging large amounts of unlabeled traffic data [33], [34], [45]. ET-BERT [34], for instance, converts raw bytes into language-like tokens for pre-training, and then fine-tune the pre-trained model for specific downstream classification tasks. PEAN [46] is a multimodal pre-training-based framework that jointly models raw bytes and packet lengths to classify encrypted traffic. Although these methods successfully extract rich features from raw traffic, they treat each traffic flow independently, neglecting the interrelations inherent in traffic flows.

C. GNN-Based Methods

Recently, an increasing number of studies have leveraged GNNs to extract topological characteristics of traffic due to their powerful ability to capture structural patterns from graph format data. Hu et al. [35] introduce the Traffic Interaction Graph (TIG) to represent each single flow based on its packet length sequence to capture implicit features in bidirectional client-server interactions, turning DApp fingerprinting into a graph classification problem. Hu et al. [35] construct a flow graph from the initial several interactive packets for each flow, adopt graph2vec algorithm to learn its representation and classify the representations with Random Forest. Zhang et al [38] propose a byte-level traffic graph construction approach that uses the GNN to embed packet header and payload bytes separately and then fuses them together to generate an overall representation. Lo et al. [36] construct a global IP-based graph for all flows where IP addresses and ports are mapped as nodes, traffic flows are mapped as the graph's edges and edge attributes are initialized with statistical traffic features. It effectively transfers the encrypted traffic classification problem into an edge classification problem. Building on this, Duan et al. [37] propose a Dynamic Line Graph Neural Network (DLGNN)-based method with semi-supervised learning. It transforms traffic data into a series of spatio-temporal graphs and employs dynamic GNN to capture the evolving of interaction between IP pairs in each snapshot. Wang et al. [39] introduce a Heterogeneous Temporal Graph (HTGraph) to represent the dynamic feature series of network flows. The HTGraph is based on prior knowledge about feature types and correlations, contributing to learning general and transferable knowledge for traffic classification.

D. Differences From Existing Work

Most existing GNN-based methods for encrypted traffic classification treat hosts in the network as nodes and the communication interaction among hosts as edges. Its primary advantage lies in the ability of leveraging GNN to extract the global topology patterns of cyberspace from the host-to-host communication interactions. However, the disadvantages to this method are obvious: it relies on feature engineering to initialize the graph attributes in advance and statistical features fail to adequately capture the temporal characteristics of traffic itself. Although few cutting-edge works like [39] attempt to leverage dynamic graphs to enhancing generalization for encrypted traffic classification, they still have not eliminate the need for feature engineering.

We compare the aforementioned encrypted traffic classification methods with our work in Table I, including ML-based, DL-based and GNN-based methods. In contrast, our method gets rid of the dependence on feature engineering and directly uses the raw-byte traffic within the global IP-based graph. We fully integrate byte-packet-flow multi-granularity features based on dynamic attributed graph to generate more discriminative traffic representations.

III. Preliminaries

A. Notation

Encrypted traffic is made up of a sequence of packets transferred among hosts. Based on the level of granularity, traffic can be commonly divided into packets, flows or sessions. A flow represents a series of consecutive packets that share the same five attributes: source IP, destination IP, source port, destination port and protocol. Flows are unidirectional, while sessions refer to bidirectional flows. To learn robust temporal representations from raw-byte flows, we propose a Packet2Vec module, which is designed to convert a packet into a word-like token, similar to those used in NLP. A single packet can be viewed as a sequence of 8-bit bytes, consisting of a plaintext-based header and a ciphertext-based payload. Since each 8-bit byte corresponds to a decimal number ranging from 0 to 255, a packet at timestamp t can be represented as a vector:

$$P_t = [b^t_{h1}, b^t_{h2}, \cdots, b^t_{hn}, b^t_{p1}, \cdots, b^t_{pm}], b^t_{hi}, b^t_{pj} \in [0, 255] \quad (1)$$

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on September 18,2026 at 06:22:17 UTC from IEEE Xplore. Restrictions apply.

TABLE I
A SUMMARY OF EXISTING APPROACHES

Category	Typical Methods	Raw bytes-based	Temporal Feature	Topological Feature	Minibatch Training	Few-shot Setting
ML-based Methods	Appscanner [12]	×	×	×	✓	×
	NB-SVM [15]	×	×	×	✓	×
	EFOA [32]	×	×	×	✓	×
DL-based Methods	TSCRNN [25]	✓	✓	×	✓	×
	FS-Net [23]	×	✓	×	✓	×
	YaTC [33]	✓	✓	×	✓	✓
	ET-BERT [34]	✓	✓	×	✓	✓
GNN-based Methods	GraphDApp [35]	✓	✓	×	✓	×
	Graph2vec+RF [35]	✓	✓	×	✓	×
	E-GraphSAGE [36]	×	×	✓	×	×
	DLGNN [37]	✓	✓	✓	×	✓
	TFE-GNN [38]	✓	✓	×	✓	×
	K-GetNID [39]	×	✓	×	✓	✓
	Ours	✓	✓	✓	✓	✓

where $b^t_{h_i}$ and $b^t_{p_j}$ denote the decimal values corresponding to header and payload bytes, respectively. A single flow, therefore, can be represented as a 2-dimensional tensor:

$$flow = [P_{t_1}, P_{t_2}, \cdots, P_{t_n}] \tag{2}$$

where P_{t_i} denotes the ith packet in the flow, t_n is the timestamp for it and n is the total number of packets of the flow. In this way, the representation of a flow can be viewed as analogous to sentence embedding in NLP, where multiple word-like vectors (packets) form a flow. This transformation allows us to leverage NLP techniques to extract temporal characteristics from network flows, facilitating a deeper analysis of temporal traffic patterns.

B. Problem Definition

The goal of encrypted traffic classification is to detect and identify specific traffic categories (e.g., applications and web services) based on the traffic they generate. In this paper, we classify encrypted flows directly from raw byte-level traffic, bypassing the need for statistical features derived through feature engineering. We evaluate our model on two types of encrypted traffic classification tasks: (1) user behavior identification over VPN or Tor (e.g., video streaming, email communication, and file transfer), and (2) encrypted application classification (e.g., Facebook, Spotify, and YouTube). In our experiment, we treat each flow as a data example for training or testing. Our objective is to build an end-to-end classification framework $f(flow_i)$ that predicts the correct label for each flow.

IV. METHODOLOGY

A. Overall Framework

In this section, we systematically introduce the proposed method, BPF-DAG, a dynamic attributed graph-based framework that integrates multi-granularity traffic features across the byte, packet, and flow levels. The architecture comprises three core modules: raw information extraction, flow representation learning, and multi-granularity feature fusion. In the first module, we directly extract topological and temporal information from raw traffic data. At the topological level, we construct a global directed IP-based graph by representing host communications as directed edges between IP-port pairs. At the temporal level, we extract fixed-length header byte sequences from individual packets, preserving essential byte-level information. The second module learns two complementary representations: temporal representations are obtained using a Transformer encoder that models sequential dependencies within packet sequences, while topological representations are learned via a custom graph neural network named Directed Edge Sample and Aggregate (DiESAGE), which propagates messages along directed edges in the graph. Edge attributes are initialized with temporal features and dynamically updated during training to reflect evolving flow representations. In the third module, we fuse the temporal and topological representations via linear interpolation, enabling the model to generate enriched, discriminative embeddings for flow classification. To efficiently support joint training of the Transformer and GNN under memory constraints, we incorporate a Memory Bank mechanism that maintains and selectively updates temporal representations across training iterations. Figure 1 provides an overview of our proposed end-to-end BPF-DAG method.

B. Raw Information Extraction

Instead of relying on feature engineering, we directly extract information from raw encrypted traffic as input for subsequent modules. This approach not only maximizes the information available from the raw traffic but also enables an end-to-end traffic classification framework. Given that encryption technology transforms packet payloads into pseudorandom values, we focus exclusively on using the raw plaintext information from the packet headers.

1) Temporal Information Extraction: At the byte level of a flow, each 8-bit byte can be encoded as a decimal number between 0 and 255. At the packet level, each packet can be transformed into a fixed-length vector, as explained in the Preliminaries section. Only the header bytes are used, while the encrypted payload bytes are discarded. The header contains a rich set of discriminative fields that provide key details about the flow, such as packet length, timestamp, and protocol. The vector length of packet header is limited to M bytes, with any excess being truncated. If the header is shorter than M bytes,

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on September 18,2026 at 06:22:17 UTC from IEEE Xplore. Restrictions apply.

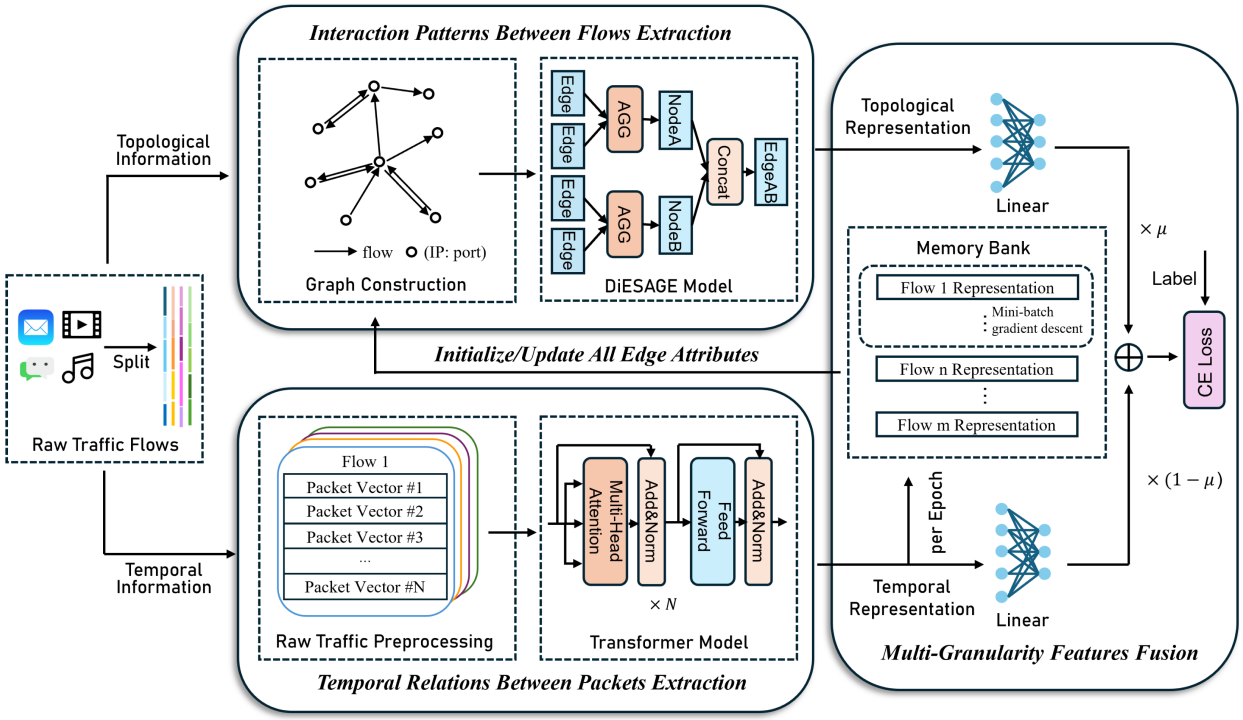


Fig. 1. The overview of proposed BPF-DAG Framework. The lower part illustrates how raw byte packet sequences are input into a Transformer to learn temporal representations. The upper part shows that network connectivity information, combined with the learnable temporal representations, is used to construct graphs, which are then processed by DiESAGE to capture topological representations. The left part details the multi-granularity feature fusion process. During the joint training of DiESAGE and the Transformer, the graph is dynamically updated by iteratively assigning edge attributes based on the evolving temporal representations.The Memory Bank technique is used to decouple the training batch size from the total number of edges. The specific module configurations and parameters are presented in Figure 4 and Table III.

it is padded with zeros. Thus, a packet at timestamp t can be encoded as a vector:

$$P_t = [b^t_{h1}, b^t_{h2}, \cdots, b^t_{hM}], b^t_{hi} \in [0, 255] \tag{3}$$

At the flow level, each flow is composed of a sequence of time-related packets. We take the first N adjacent packets, which form a sequence of N M-dimensional vectors, as the initial representation of the flow. If the flow has fewer than N packets, it is padded with zero vectors. Ultimately, a flow is represented as a two-dimensional tensor of size (N, M):

$$flow_i = [P_1, P_2, \cdots, P_N], P_i \in \mathbb{R}^M \tag{4}$$

In fact, this initial flow representation closely resembles a sentence composed of word vectors. Hence, NLP techniques can naturally be applied to learn the temporal representation of the flow, as discussed in the following section.

2) Topological Information Extraction: Computer networks fundamentally exist as communication graphs, where each host acts as a node, and the communication between hosts represents the links between nodes. From the perspective of a single flow, it can be modeled as a directed edge with two nodes in the communication graph. Four flow fields are extracted to identify the directed edge: source IP address, source port, destination IP address, and destination port. The first two fields are combined into a 2-tuple (source IP address, source port) to represent the source node, while the latter two fields represent the destination node.

To mine the topological information of host-to-host communications, we construct a global IP-based flow interaction graph. The previous temporal information will be integrated into the graph. The initial flow representation $flow_i$ will be fed into a neural network with learnable parameters to generate its temporal representation, which is utilized to initialize its corresponding edge attribute. Before training, each edge attribute is initialized using the corresponding temporal representation, computed by randomly initialized trainable parameters. During the training process, the edge attributes are dynamically updated and optimized.

In this way, the encrypted traffic classification task is naturally transformed into a directed edge classification problem.

C. Flow Representation Learning

In this module, based on the temporal and topological information extracted from the raw traffic, we introduce a sequence-based model and a novel graph neural network to learn the temporal and topological representations respectively.

1) Learning Temporal Representations With Transformer: In the Preliminaries Section, we encoded a flow as a sequence of packet vectors, represented as a two-dimensional tensor. The relationship between a flow and its packets is analogous to that of a sentence and its words. Building on this analogy, we utilize the Transformer network [47] to learn temporal representations of traffic flows. We use the same architecture as the classic Transformer encoder.

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on September 18,2026 at 06:22:17 UTC from IEEE Xplore. Restrictions apply.

The classic Transformer architecture, initially designed to address sequence-to-sequence (Seq2Seq) tasks [48], is composed of an encoder block and a decoder block. For our purposes, we employ only the encoder to process the input traffic sequences and generate temporal flow representations. The encoder consists of multiple identical blocks, each containing two primary sublayers: a multi-head self-attention mechanism and a position-wise feed-forward network (FFN). In the first layer, the self-attention mechanism enables each position in the input sequence to pay attention to other positions, capturing time-related characteristics throughout the sequence. For an input sequence $flow_i \in \mathbb{R}^{N\times M}$, each self-attention head computes a query matrix Q, key matrix K and value matrix V through the following linear transformations:

$$Q = Z\mathbf{W}^Q, K = Z\mathbf{W}^K, V = Z\mathbf{W}^V \tag{5}$$

where $\mathbf{W}^Q, \mathbf{W}^K, \mathbf{W}^V \in \mathbb{R}^{M\times d}$ are trainable parameter matrices, $Q, K, V \in \mathbb{R}^{N\times d}$ and d is the dimension of the latent representation. The output of the i th self-attention head is computed as:

$$Head_i = Attention(Q_i, K_i, V_i) = soft\max\left(\frac{Q_i {K_i}^T}{\sqrt{d}}\right) V_i \tag{6}$$

The attention mechanism is simultaneously applied across several heads, and then the outputs from each head are concatenated and subjected to a final linear transformation:

$$MultiHead(Q, K, V) = \text{CONCAT}(Head_1, \cdots, Head_n)\mathbf{W}^O \tag{7}$$

In the second layer, a FFN is employed independently and uniformly to each position. It is composed of two linear transformations with a ReLU activation function in between:

$$FFN(x) = \max(0, x\mathbf{W}_1 + b_1)\mathbf{W}_2 + b_2 \tag{8}$$

Additionally, residual connection and layer normalization operations are applied to both sublayers. These techniques enhance gradient flow and facilitate stable training, while supporting deeper network architectures.

$$Z' = LayerNorm(Z + MultiHead(Q, K, V)) \tag{9}$$

$$X_{\text{temporal}} = LayerNorm(Z' + FFN(Z')) \tag{10}$$

The output of the Transformer, denoted as $X_{\text{temporal}} \in \mathbb{R}^{N\times D}$, is referred to as the temporal representation of the flow.

2) Learning Topological Representations With DiESAGE: To further capture the topological characteristics of traffic through interaction among hosts, we introduce a novel graph neural network named Directed Edge Sample and aggregate (DiESAGE). GraphSAGE (Graph Sample and Aggregate) [31] is a powerful GNN framework designed for generating node embeddings in large graphs, primarily tailored for node classification tasks. However, the traditional GraphSAGE model is specifically designed for learning node embeddings. The existing E-GraphSAGE method [36] is not suited for generating representations for directed edges. To address this limitation, our proposed DiESAGE introduces several modifications to support directed edge embeddings.

The inputs to DiESAGE consist of edge and node features. The edge features are initialized with the temporal representations X_{temporal}, which are dynamically updated at each epoch during the training process. Meanwhile, the node features are initialized with constant vectors $\{1 \cdots 1\}$, where the dimension of each node vector matches that of the edge features.

Algorithm 1 DiESAGE Directed Edge Embedding

Input: Graph $\mathcal{G}(\mathcal{V}, \mathcal{E})$; input edge attributes $\{\mathbf{e}_{uv}, \forall uv \in \mathcal{E}\}$; input node attributes $\{n_v, \forall v \in \mathcal{V}\} = \{1, \ldots, 1\}$; depth K

Output: Directed edge embeddings $\{z_{uv}, \forall uv \in \mathcal{E}\}$

$\mathbf{h}_0 \leftarrow n_v, \forall v \in \mathcal{V}$
$\mathbf{e}^0_{uv} \leftarrow e_{uv}, \forall uv \in \mathcal{E}$
for $k \leftarrow 1$ **to** K **do**
  for $u \in \mathcal{V}$ **do**
    $\mathbf{h}^k_{\mathcal{N}(u)} \leftarrow \text{AGG}\left(\left\{\mathbf{e}^{k-1}_{uv}, \forall v \in \mathcal{N}(u), uv \in \mathcal{E}\right\}\right)$
    $\mathbf{h}^k_u \leftarrow \sigma\left(\mathbf{W}^k \cdot \text{CONCAT}\left(\mathbf{h}^{k-1}_u, \mathbf{h}^k_{\mathcal{N}(u)}\right)\right)$
  end for
end for
for $uv \in \mathcal{E}$ **do**
  $z_{uv} = \text{CONCAT}\left(\mathbf{h}^K_u, \mathbf{h}^K_v\right), uv \in \mathcal{E}$
end for
return z_{uv}

At each iteration, instead of aggregating information from neighboring nodes, DiESAGE aggregates features from directed edges pointing from the node to its neighbors, and subsequently updates the node's features based on the aggregated information. At the kth layer, the aggregated representation $\mathbf{h}^k_{\mathcal{N}(u)}$ is computed using a differentiable aggregation function AGG, defined as:

$$\mathbf{h}^k_{\mathcal{N}(u)} = \text{AGG}\left(\left\{\mathbf{e}^{k-1}_{uv}, \forall v \in \mathcal{N}(u), uv \in \mathcal{E}\right\}\right) \tag{11}$$

Here, $\{v \in \mathcal{N}(u), uv \in \mathcal{E}\}$ represents the set of directed edges pointing from source node u to its neighbor nodes, and $\mathbf{e}^{k-1}_{uv}$ denotes the edge features at the $k-1$th layer. The aggregation function AGG calculates the mean of $\mathbf{e}^{k-1}_{uv}$. Next, the aggregated representation $\mathbf{h}^k_{\mathcal{N}(u)}$ of the neighbor edges is concatenated with the node's previous representation $\mathbf{h}^{k-1}_u$. After applying a trainable weight matrix $\mathbf{W}^k$ and an activation function, the node representation at the kth layer is updated as follows:

$$\mathbf{h}^k_u = \sigma\left(\mathbf{W}^k \cdot \text{CONCAT}\left(\mathbf{h}^{k-1}_u, \mathbf{h}^k_{\mathcal{N}(u)}\right)\right) \tag{12}$$

At depth K, the final representation of the edge $\mathbf{e}^K_{uv}$ is computed by concatenating representations of node u and v as follows:

$$X_{\text{topological}} = \text{CONCAT}\left(\mathbf{h}^K_u, \mathbf{h}^K_v\right), uv \in \mathcal{E} \tag{13}$$

The output of DiESAGE, referred to as the topological representation $X_{\text{topological}} \in \mathbb{R}^{D\times d}$, captures the topological information of the communication graph. Figure 3 illustrates the process of message propagating through nodes and edges in 1-hop scenario. Algorithm 1 presents the DiESAGE algorithm for directed edge embedding, and the relevant symbols are summarized in Table II.

D. Multi-Granularity Features Fusion

After capturing both the temporal and topological representations of the traffic, we fuse them via linear interpolation to

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on September 18,2026 at 06:22:17 UTC from IEEE Xplore. Restrictions apply.

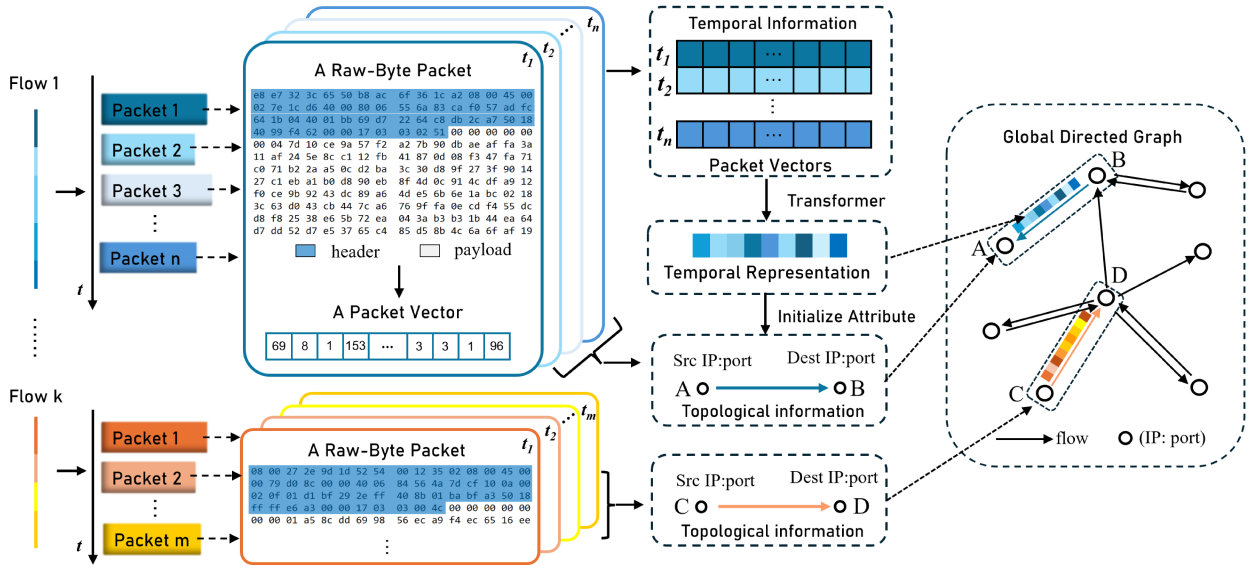


Fig. 2. The schematic illustration of flow preprocessing.

TABLE II
SUMMARY OF SYMBOLS AND DESCRIPTIONS

Symbols	Descriptions
$\mathcal{E}$	The set of directed edges in Graph $\mathcal{G}$
$\mathcal{V}$	The set of nodes in Graph $\mathcal{G}$
$\mathbf{W}^k$	The trainable weight matrix at the k-th layer
$\mathbf{h}_u^k$	The embeddings of nodes at the k-th layer
$\mathbf{e}_{uv}^k$	The embeddings of edges at the k-th layer
$\mathbf{z}_{uv}$	The final directed edge embeddings
AGG	The differentiable aggregator function
$\mathcal{F}$	Input samples of raw byte-based flows
$\mathcal{N}(u)$	The set of neighbors of node u
σ	The ReLU activation function
X_{temporal}	Temporal representations extracted with Transformer
X_{temporal}'	A sampled mini-batch of temporal representations
$X_{\text{topological}}$	Topological representations extracted with DiESAGE
Z_{temporal}	Low-dimensional temporal prediction vectors
$Z_{\text{topological}}$	Low-dimensional topological prediction vectors
Z_{fusion}	Final features fusion representations
$\theta_{\text{BPF-DAG}}$	The parameters of model BPF-DAG

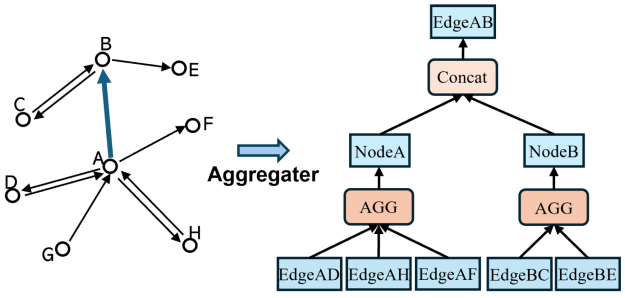


Fig. 3. The illustration of the DiESAGE model in 1-hop propagation scenario.

maximize the extracted traffic information. The cross-entropy loss is employed over labeled edges for jointly optimizing both the Transformer and DiESAGE models. Additionally, a technique called Memory Bank is employed to enable mini-batch gradient descent when training the GNN. The pseudocode of our method is presented in Algorithm 2 and the relevant symbols are summarized in Table II.

Algorithm 2 Pseudocode of the BPF-DAG Algorithm

Input: Graph $\mathcal{G}(\mathcal{V}, \mathcal{E})$; raw-byte flows $\mathcal{F}$; input edge attributes $\{\mathbf{e}_{uv}, \forall uv \in \mathcal{E}\}$; depth K; number of training epochs T; linear interpolation coefficient μ; traffic label Y_{label}
Output: Parameters of BPF-DAG model $\theta_{\text{BPF-DAG}}$

for $epoch \leftarrow 1$ **to** T **do**
  // Update attributes of all edge using Transformer after each epoch
  $X_{\text{temporal}} \leftarrow Transformer_{\text{Encoder}}(\mathcal{F})$
  Update edge features $\{e_{uv}, \forall uv \in \mathcal{E}\}$ with X_{temporal}
  // Set a Memory Bank to track all temporal representations
  for $\mathcal{F}_{\text{mini-batch}} \in \mathcal{F}$ **do**
    // Update a subset of edges corresponding to the sampled mini-batch with optimized Transformer
    $X_{\text{temporal}}' \leftarrow Transformer_{\text{Encoder}}(\mathcal{F}_{\text{mini-batch}})$
    Update edge features $\{e_{uv}, \forall uv \in \mathcal{E}_{\text{mini-batch}}\}$ with X_{temporal}'
    $X_{\text{temporal}} \leftarrow \{e_{uv}, \forall uv \in \mathcal{E}\}$
    $X_{\text{topological}} = DiESAGE(e_{uv}), \forall uv \in \mathcal{E}$
    // Map representations to low-dimensional prediction vectors
    $Z_{\text{temporal}} \leftarrow softmax(\mathbf{W}_1 X_{\text{temporal}})$
    $Z_{\text{topological}} \leftarrow softmax(\mathbf{W}_2 X_{\text{topological}})$
    // Generate final representations with linear interpolation
    $Z_{\text{fusion}} \leftarrow (1-\mu) Z_{\text{temporal}} + \mu Z_{\text{topological}}$
    $\mathcal{L}oss \leftarrow CrossEntropy(Z_{\text{fusion}}, Y_{\text{label}})$
    $\theta_{\text{BPF-DAG}} \leftarrow Adam(\mathcal{L}oss)$
  end for
end for
return $\theta_{\text{BPF-DAG}}$

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on September 18,2026 at 06:22:17 UTC from IEEE Xplore. Restrictions apply.

TABLE III
DETAILED ARCHITECTURE AND PARAMETER STATISTICS OF THE BPF-DAG MODEL

Layer (type)	Input Shape	Output Shape	Param #
Seq Model (Transformer)	[-1, 10, M]	[-1, 128]	**1,058,800**
PositionalEncoding	[-1, 10, M]	[-1, 10, M]	0
4 × TransformerEncoderLayer	[-1, 10, M]	[-1, 10, M]	∼1,051,000
MultiheadAttention	[-1, 10, M]	[-1, 10, M]	14,640 × 4
FeedForward (Linear1+Linear2)	[-1, 10, M]	[-1, 10, M]	(124,928 + 122,940) × 4
LayerNorm + Dropout	[-1, 10, M]	[-1, 10, M]	240 × 4
Linear (fc)	[-1, M]	[-1, 128]	7,808
Classifier	[-1, 128]	[-1, num_classes]	**1,032**
Linear	[-1, 128]	[-1, num_classes]	1,032
GNN (DiESAGE)	[edges, 128]	[edges, num_classes]	**133,640**
SAGE Layer 0	[edges, 128]	[edges, 128]	65,792
W_msg (Linear)	[edges, 256]	[edges, 128]	32,896
W_apply (Linear)	[edges, 256]	[edges, 128]	32,896
SAGE Layer 1	[edges, 128]	[edges, 128]	65,792
W_msg (Linear)	[edges, 256]	[edges, 128]	32,896
W_apply (Linear)	[edges, 256]	[edges, 128]	32,896
MLP Predictor (W)	[edges, 256]	[edges, num_classes]	2,056
Total Parameters	–	–	**1,193,472**

1) Interpolating Topological and Temporal Representations: To fuse the topological and temporal representations, we linearly interpolate X_{temporal} and $X_{\text{topological}}$, then feed them into a linear layer to obtain the final prediction output. During our experiments, we found that applying an auxiliary transformation before classification improves the overall accuracy. Specifically, we map the temporal and topological representations to low-dimensional prediction vectors through the following auxiliary function:

$$Z_{\text{temporal}} = softmax(\mathbf{W}_1 X_{\text{temporal}}) \tag{14}$$

$$Z_{\text{topological}} = softmax(\mathbf{W}_2 X_{\text{topological}}) \tag{15}$$

where $\mathbf{W}_1, \mathbf{W}_2 \in \mathbb{R}^{d \times p}$ denote trainable weight matrices, and p represents the number of traffic categories. Next, we apply linear interpolation to obtain the final prediction vector:

$$Z_{\text{fusion}} = (1-\mu) Z_{\text{temporal}} + \mu Z_{\text{topological}} \tag{16}$$

The coefficient μ is a hyperparameter controlling the balance between temporal and topological information. When $\mu = 0$, only the temporal representation is used, ignoring the topological features of network traffic. Conversely, $\mu = 1$ fully utilizes the topological features, but some temporal characteristics may be lost owing to the graph message-passing process. A balanced value of $0 < \mu < 1$ optimally fuses both the temporal and topological predictions. The cross-entropy loss is employed as the objective function over labeled directed edges:

$$L = CrossEntropy(Z_{\text{fusion}}, Y_{\text{label}}) \tag{17}$$

2) Updating Graph Attributes With Memory Bank: Throughout training, edge attributes are dynamically updated using X_{temporal} at each epoch, allowing the temporal representation of each flow to remain optimal during training.

Traditional GNN models typically use full-batch gradient descent, which is unsuitable for our BPF-DAG model for two reasons. First, the scale of the Internet is vast, involving a massive number of IP addresses, which likely makes the graph quite large. Second, simultaneously optimizing both the Transformer and DiESAGE models may lead to memory limitations on the GPU. To address these challenges, we adopt a Memory Bank (MB) technique inspired by contrastive learning methods [49], [50]. MB decouples the mini-batch size from the total number of graph edges by storing their representations.

Specifically, we set a MB to keep track of the temporal representations X_{temporal} of both labeled and unlabeled flow samples. At the start of each epoch, all X_{temporal} representations are computed using the current Transformer parameters, and the MB is updated. During each mini-batch iteration, we sample a mini batch from the graph edges, recompute their representations with the current Transformer, and update the corresponding entries in the MB. These updated representations, denoted as X_{temporal}', are then fed into the DiESAGE model. The cross-entropy loss for the mini-batch is then computed and backpropagated.

Since X_{temporal} in the MB are computed at different iterations within each epoch, they may become inconsistent. To mitigate this, we use a relatively small learning rate for the Transformer to improve the consistency of stored representations in the MB.

V. EXPERIMENTS

A. Experiment Setup

1) Datasets: To comprehensively evaluate the effectiveness of BPF-DAG, we conducted experiments on six public encrypted traffic datasets, including the ISCX VPN-nonVPN dataset [51], the ISCX Tor-nonTor dataset [52], the MIRAGE-2019 dataset [53] and the MIRAGE-2024 dataset [54].

The VPN-nonVPN dataset contains encrypted traffic routed through virtual private networks (VPNs) and regular encrypted traffic, both of which encompass six traffic categories: P2P, VoIP, etc. VPNs are commonly used to bypass censorship and access restricted services, and the use of tunnel technology complicates the identification of applications running through

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on September 18,2026 at 06:22:17 UTC from IEEE Xplore. Restrictions apply.

TABLE IV
RECOMMENDED HYPERPARAMETER CONFIGURATION FOR BPF-DAG

Groups	Hyperparameter	Description	Value
Model Setup	n_e	The number of encoders in Transformer.	4
	n_h	The number of headers for multi-head self-attention.	5
	K	The number of layers in DiESAGE.	2
	p	The dropout rate in each layer.	0.2
Traffic Pre-Processing	N	The number of the first few packets truncated in each flow.	10
	M	The number of the first few raw bytes truncated in each packet.	60/200
	D_T	The embedding size of temporal representations.	128
	D_S	The embedding size of topological representations.	128
Training process	μ	The linear interpolation coefficient of topological and temporal representations.	0.9/0.4
	LR_T	The initial learning rate of the optimizer for Transformer.	1e-5
	LR_S	The initial learning rate of the optimizer for DiESAGE.	1e-3
	BATCH_SIZE	The number of training examples during each iteration.	128

VPNs. Similarly, the Tor-nonTor dataset includes traffic collected via onion routers (Tor) and regular encrypted traffic, both featuring eight types of traffic: browsing, audio streaming, chat, video streaming, mail, VoIP, P2P, and file transfer. Tor anonymizes internet activities by encrypting and routing traffic through a distributed network, which adds complexity to traffic classification. The MIRAGE-2019 dataset is a publicly available, human-generated mobile traffic dataset designed for encrypted mobile app traffic analysis. It includes traffic from 40 Android applications across 16 app categories, focusing on common functionalities such as service registration, login, and regular interactions. The release of the MIRAGE-2024 dataset aims to facilitate further research and development in mobile network traffic analysis, focusing on interactive, multi activity apps and activity-level analysis.

2) Data Preprocessing: Firstly, we remove packets irrelevant to the transmitted content of traffic, such as Address Resolution Protocol (ARP) and Internet Control Message Protocol (ICMP) packets. Then, we use SplitCap tool to split the raw traffic datasets into unidirectional flows and each flow is taken as a sample for training or testing. To address the scarcity of flow examples in some datasets, especially Tor dataset, we augment the flow samples by splitting longer flows into shorter ones. The number of splitted flows from the original long flow is no more than 10. Finally, we partition each dataset into training, validation, and testing sets in a 7:1:2 ratio.

3) Implementation Details and Evaluation Metrics: During the raw traffic pre-processing stage of BPF-DAG, we truncate the first 60 and 200 bytes of each raw packet for the ISCX and MIRAGE datasets, respectively. The MIRAGE datasets lacks original pcap files and comprises only transport-layer payload bytes, without the associated packet header information. Therefore, we use only the encrypted payload bytes of the MIRAGE datasets as input. In our experiments, only the first 10 packets of each flow are retained for all datasets, and the remaining packets are discarded. This retained length is considerably shorter than that used in [8], [12], and [22], where some flows include up to 5,000 packets. For training, we utilize the encoder of the Transformer and a two-layer DiESAGE model to implement the BPF-DAG method. The Adam optimizer with a learning rate scheduler is applied, with initial learning rates of 1e-3 for the DiESAGE model and 1e-5 for

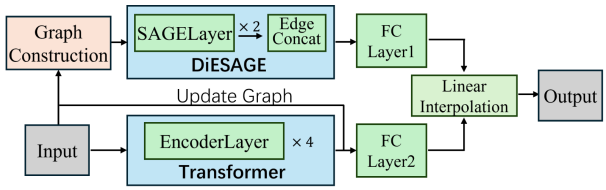


Fig. 4. The module-level overview of BPF-DAG.

the Transformer model. The batch size is set to 64, and training runs for 100 epochs. The model is implemented in PyTorch 2.0.1 and trained on a single NVIDIA RTX 4060 GPU. We run each experiment independently 5 times and average the results to reduce the impact of any variability or outliers. Table IV shows the hyperparameters used in BPF-DAG and the recommended values. We provided a module-level overview of the BPF-DAG architecture in Figure 4, a detailed layer-level description in Table III, showing the input/output dimensions and parameter counts for each neural network layer.

To evaluate and compare the classification performance of our method, we utilize the four key metrics: Accuracy (AC), Precision (PR), Recall (RC), and F1-score (F1). They are calculated as shown below.

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN} \tag{18}$$

$$Recall = \frac{TP}{TP + FN} \tag{19}$$

$$Precision = \frac{TP}{TP + FP} \tag{20}$$

$$F1 = 2 \times \frac{Recall \times Precision}{Recall + Precision} \tag{21}$$

where TP, FP, TN, and FN represent true positive, false positive, true negative, and false negative, respectively. Considering the imbalanced class distribution of traffic types, macro averaging is applied to calculate the mean values of Accuracy, Precision, Recall, and F1-score in this paper.

$$Macro_{average} = \frac{1}{k} \sum_{i=1}^{k} m_i \tag{22}$$

where m is the classification metric, m_i is the value of classification metric m of class i, and k is the total number of traffic categories.

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on September 18,2026 at 06:22:17 UTC from IEEE Xplore. Restrictions apply.

TABLE V
EXPERIMENTAL RESULTS ON PUBLIC ISCX VPN-NONVPN DATASETS

		ISCX-VPN				ISCX-nonVPN			
Category	Model	AC	RC	PR	F1	AC	RC	PR	F1
ML-based Methods	AppScanner [12]	0.7490	0.6138	0.7153	0.6176	0.7140	0.6406	0.7298	0.6524
	CUMUL [13]	0.6586	0.5682	0.6190	0.5797	0.6255	0.5041	0.5874	0.4984
DL-based Methods	1D-CNN [21]	0.9540	0.9313	0.9472	0.9387	0.8483	0.7910	0.8264	0.8042
	2D-CNN [42]	0.8636	0.8143	0.8594	0.8324	0.8390	0.7951	0.8206	0.8059
	HAST [43]	0.9597	0.9452	0.9526	0.9489	0.8914	0.8542	0.8781	0.8660
	TSCRNN [25]	0.9535	0.9439	0.9416	0.9424	0.8471	0.8069	0.8120	0.8089
	ET-BERT [34]	0.9762	0.9631	0.9676	0.9653	0.9407	0.9201	0.9348	0.9267
	YaTC [33]	0.9851	0.9851	0.9855	0.9852	0.9511	0.8978	0.9332	0.9139
GNN-based Methods	E-GraphSAGE [36]	0.7585	0.7245	0.7285	0.7259	0.5215	0.5215	0.5379	0.5158
	TFE-GNN [38]	0.9598	0.9456	0.9627	0.9529	0.9421	0.9229	0.9334	0.9267
	Ours	**0.9912**	**0.9865**	**0.9895**	**0.9879**	**0.9637**	**0.9327**	**0.9448**	**0.9382**

TABLE VI
EXPERIMENTAL RESULTS ON PUBLIC ISCX TOR-NONTOR DATASETS

		ISCX-Tor				ISCX-nonTor			
Category	Model	AC	RC	PR	F1	AC	RC	PR	F1
ML-based Methods	AppScanner [12]	0.6529	0.5014	0.6457	0.4800	0.7288	0.5699	0.6566	0.6102
	CUMUL [13]	0.5728	0.4273	0.4120	0.3974	0.6523	0.5123	0.6015	0.5533
DL-based Methods	1D-CNN [21]	0.9336	0.9217	0.9330	0.9268	0.9380	0.8342	0.8779	0.8538
	2D-CNN [42]	0.9336	0.9220	0.9239	0.9221	0.9253	0.7864	0.8435	0.8109
	HAST [43]	0.9502	0.9401	0.9474	0.9437	0.9583	0.8530	0.9024	0.8770
	TSCRNN [25]	0.9483	0.9392	0.9400	0.9386	0.9503	0.8513	0.8938	0.8721
	ET-BERT [34]	0.9561	0.9247	0.9457	0.9339	0.9732	0.9244	0.9398	0.9320
	YaTC [33]	0.9497	0.9368	0.9474	0.9400	0.9776	0.9297	0.9528	0.9402
GNN-based Methods	E-GraphSAGE [36]	0.4647	0.3866	0.4502	0.3717	0.5764	0.5764	0.5729	0.5659
	TFE-GNN [38]	0.9163	0.8712	0.9195	0.8832	0.9756	0.9421	0.9524	0.9471
	Ours	**0.9797**	**0.9739**	**0.9772**	**0.9749**	**0.9795**	**0.9452**	**0.9558**	**0.9501**

B. Comparison With SOTA Methods

We compare BPF-DAG with a range of baselines and state-of-the-art (SOTA) methods, including (1) ML-based methods: AppScanner [12], CUMUL [13]; (2) DL-based methods: 1D-CNN [21], 2D-CNN [42], TSCRNN [25], HAST [43]; (3) pretraining-based methods: ET-BERT [34], YaTC [33]; (4) GNN-based methods: E-GraphSAGE [36], TFE-GNN [38].

From the comparison results shown in Table V, VI and VII, we can observe that BPF-DAG outperforms all baseline and SOTA methods across all datasets. The experimental results indicate that BPF-DAG outperforms existing SOTA methods by 1.4%, 1.2%, 2.3%, 0.2%, 7.7% and 3.1% in terms of accuracy.

ML-based methods in our experiments perform poorly, largely because they only use the packet length sequence as input features, instead of raw traffic bytes, which is not reliable for encrypted traffic classification. In our experimental setup, the number of packets in each flow is generally small, which is insufficient for ML-based methods to learn discriminative features.

SOTA pretraining-based methods like ET-BERT and YaTC perform well across the datasets. Encryption and anonymity techniques make it challenging for DL-based methods to analyze traffic payloads directly. Pretraining-based methods outperform other DL methods, underscoring the value of pretraining on large-scale traffic data. However, BPF-DAG achieves slightly higher accuracy than these pretraining methods, even without using traffic payloads or pretraining techniques. This highlights our method's ability to avoid the computing overhead of pretraining while reducing the cost associated with processing payload bytes in each flow sample.

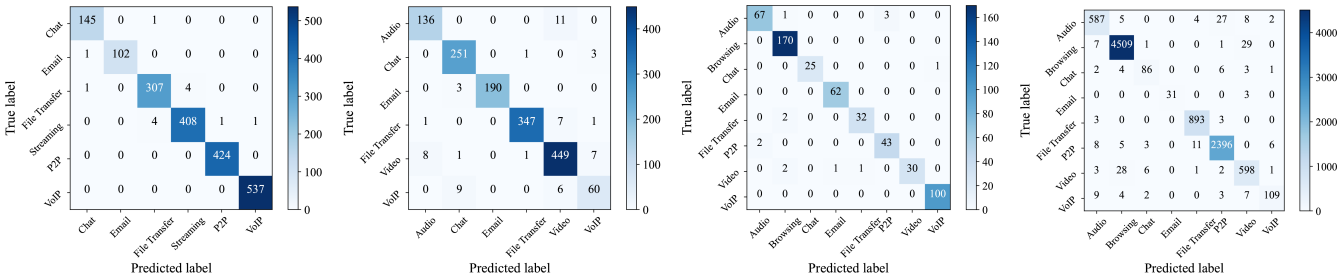
The performance of GNN-based methods, such as TFE-GNN and E-GraphSAGE, varies with their specific graph construction approaches. E-GraphSAGE relies on manually extracted statistical features to initialize edge attributes, lacking an end-to-end framework. Furthermore, its use of only a few low-dimensional statistical features limits its ability to capture discriminative features, resulting in less-than-ideal and unreliable performance. TFE-GNN constructs a byte-level traffic graph where raw bytes are converted into nodes. However, it treats each flow independently, without considering flow-level interaction patterns, which is not reliable enough. In contrast, BPF-DAG efficiently integrates byte-level, packet-level and flow-level traffic features, enabling a more reliable and accurate framework.

Figure 5 and 6 present the visualization of normal and normalized confusion matrices on all ISCX datasets, respectively. From the normal confusion matrices, it is clear that the distribution of different traffic types within each dataset is imbalanced, especially in the ISCX-nonTor dataset. Despite

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on September 18,2026 at 06:22:17 UTC from IEEE Xplore. Restrictions apply.

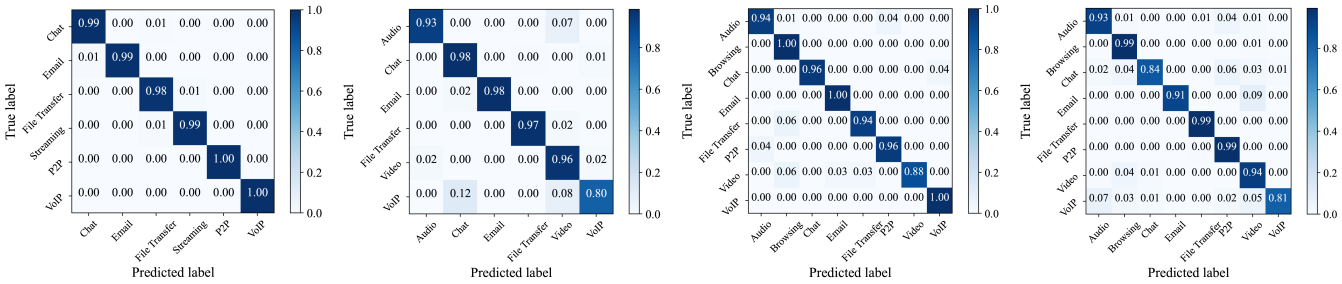
TABLE VII
EXPERIMENTAL RESULTS ON MIRAGE-2019 AND MIRAGE-2024 DATASETS

		MIRAGE-2019				MIRAGE-2024			
Category	Model	AC	RC	PR	F1	AC	RC	PR	F1
ML-based Methods	AppScanner [12]	0.6503	0.5820	0.6722	0.6239	0.5792	0.5162	0.5761	0.5445
	CUMUL [13]	0.5955	0.5209	0.6056	0.5601	0.5573	0.449	0.5758	0.5046
DL-based Methods	1D-CNN [21]	0.7276	0.6820	0.6956	0.6887	0.7066	0.6888	0.7442	0.7154
	2D-CNN [42]	0.6950	0.6440	0.6796	0.6613	0.6863	0.6606	0.7336	0.6952
	HAST [43]	0.7442	0.6935	0.707	0.7002	0.6995	0.6648	0.7127	0.6879
	TSCRNN [25]	0.7457	0.7003	0.7028	0.7015	0.7220	0.7031	0.7428	0.7224
	ET-BERT [34]	0.7140	0.6600	0.7502	0.7022	0.7500	0.7045	0.7154	0.7099
	YaTC [33]	0.8314	0.7944	0.8225	0.8082	0.7935	0.7728	0.8192	0.7953
GNN-based Methods	E-GraphSAGE [36]	0.7033	0.6352	0.6897	0.6614	0.6642	0.5760	0.6627	0.6163
	TFE-GNN [38]	0.7558	0.7041	0.7688	0.7350	0.7345	0.6896	0.7272	0.7079
	Ours	**0.9083**	**0.8929**	**0.8999**	**0.8964**	**0.8247**	**0.7988**	**0.8400**	**0.8189**



(a) ISCX-VPN (b) ISCX-nonVPN (c) ISCX-Tor (d) ISCX-nonTor

Fig. 5. Confusion matrices of BPF-DAG on four datasets.



(a) ISCX-VPN (b) ISCX-nonVPN (c) ISCX-Tor (d) ISCX-nonTor

Fig. 6. Normalized confusion matrices of BPF-DAG on four datasets.

the imbalance, BPF-DAG delivers satisfactory classification results across all datasets. Additionally, although the ISCX-Tor dataset has considerably fewer samples compared to the others, BPF-DAG outperforms SOTA methods by a significant margin.

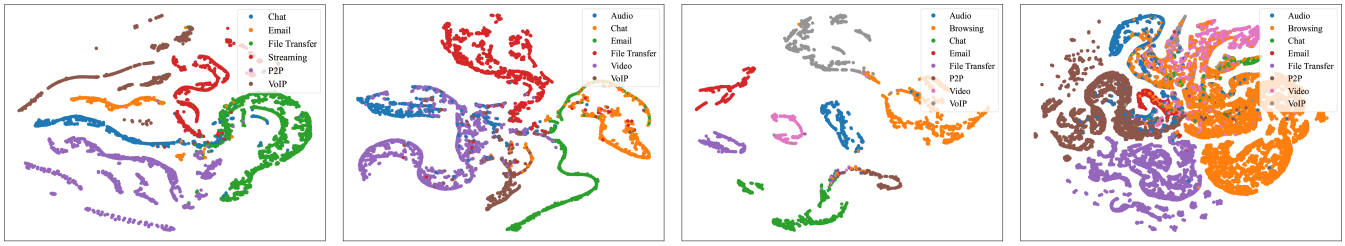
C. Ablation Studies

To further assess the contribution of each component in BPF-DAG and verify the reliability of our method, we perform an ablation study on the ISCX-VPN dataset. The results are presented in Table VIII. We denote the extraction of temporal traffic information as 'T' and topological traffic information as 'S'. More specifically, 'w/o T' in Table VIII refers to the setup where traffic's statistical features are used to initialize the graph attributes instead of automatically extracting temporal features. This evaluates the impact of the packet-level temporal features and byte-level raw information. 'w/o S' refers to extracting temporal features from each flow independently, without considering the topological relationships among flows.

TABLE VIII
ABLATION STUDY OF BPF-DAG ON ISCX-VPN DATASETS

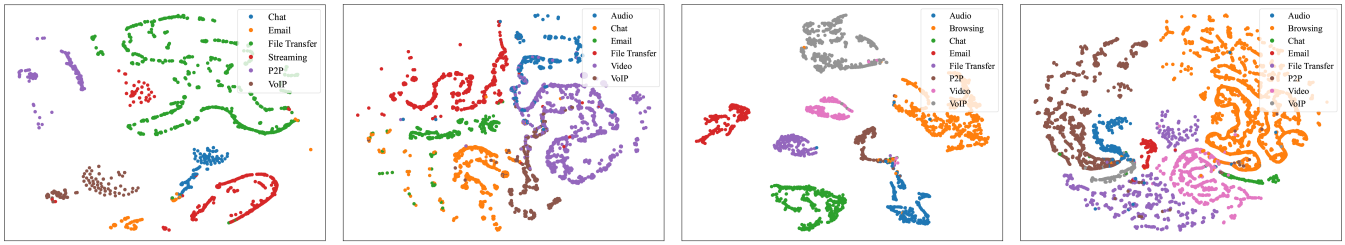
Model	AC	PR	RC	F1
Ours w/o T	0.8440	0.7778	0.8414	0.7940
Ours w/o S	0.9793	0.9686	0.9669	0.9676
w AttnLSTM	0.9886	0.9876	0.9826	0.9846
w CNN	0.9587	0.9315	0.9500	0.9387
w Concatenation	0.9742	0.9623	0.9616	0.9617
w Average	0.9587	0.9534	0.9391	0.9457
Ours(default)	0.9912	0.9865	0.9895	0.9879

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on September 18,2026 at 06:22:17 UTC from IEEE Xplore. Restrictions apply.



(a) ISCX-VPN (b) ISCX-nonVPN (c) ISCX-Tor (d) ISCX-nonTor

Fig. 7. t-SNE Visualization of ablation study without using topological features on four datasets.



(a) ISCX-VPN (b) ISCX-nonVPN (c) ISCX-Tor (d) ISCX-nonTor

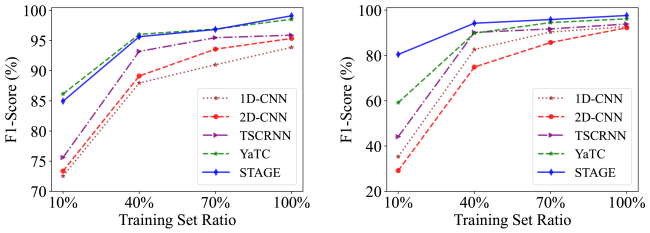
Fig. 8. t-SNE Visualization of the proposed BPF-DAG method on four datasets.

This setup tests the impact of the flow-level features. Additionally, we assess the effectiveness of alternative temporal feature extractor and feature fusion operations, respectively. We replace the 'Transformer' module with 'Attention-based LSTM' (AttnLSTM) and 'CNN' module, to see how these different architectures affect performance in capturing temporal representations. We substitute the linear interpolation operation used to fuse temporal and topological representations with concatenation and average operations to decide the optimal fusion operation.

Based on the ablation study results, we can derive the following conclusions:

- Both temporal and topological information contribute to improving classification performance. In contrast to treating each flow as an isolated entity, fully leveraging and capturing interrelationship between flows helps improve accuracy. Moreover, automatic feature extraction from raw traffic outperforms traditional feature engineering. Automatic extraction not only eliminates the need for manually designing statistical features, but also increases accuracy and reliability of the model.
- Several operations for fusing temporal and topological features were tested, such as concatenation, average, and linear concatenation. The results show that linear interpolation is the most effective for combining the two representations.
- The 'AttnLSTM' module provides classification accuracy comparable to the default Transformer module, indicating its ability to capture temporal features effectively. The CNN module, however, performs less effectively, likely because it lacks the capacity to capture the sequential, time-related dependencies.

To conclude, the ablation studies confirm that the byte-level, packet-level and flow-level traffic features all contribute to the model's accuracy and reliability. Besides, to visualize the improvement brought by flow-level topological features, we use t-distributed stochastic neighbor embedding (t-SNE) to map the final representations to a lower-dimensional space, both with and without topological features, and visualize them as two-dimensional images, as shown in Figure 7 and 8.



(a) ISCX-VPN (b) ISCX-Tor

Fig. 9. Comparison results in few-shot settings on ISCX-VPN and ISCX-Tor datasets.

D. Few-Shot Analysis

To evaluate the performance of BPF-DAG in few-shot scenarios, we designed comparison experiments using varied proportions of data on the ISCX-VPN and ISCX-Tor datasets. Specifically, we randomly selected 10%, 40%, and 70% of the original training samples and conducted classification under the same conditions. We compared BPF-DAG with several baselines as well as SOTA methods, with the comparsion results presented in Figure 9.

The results show that BPF-DAG is the least affected by reduced training data and consistently outperforms other methods across all data proportions. In contrast, common deep learning-based methods (e.g., 1D-CNN, 2D-CNN, TSCRNN) suffer significant performance degradation when the proportion of training data is reduced to 10%. However, the

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on September 18,2026 at 06:22:17 UTC from IEEE Xplore. Restrictions apply.

TABLE IX
MODEL FLOPS AND PARAMETERS

Model	Parameters (M)	FLOPs (M)
1D-CNN	3.7E+00	1.3E+01
2D-CNN	2.2E-01	6.9E+00
TSCRNN	8.5E-01	4.3E+01
HAST	4.7E-01	3.3E+02
TFE-GNN	4.4E+01	2.2E+03
ET-BERT	8.6E+01	1.1E+04
YaTC	1.8E+01	9.1E+02
BPF-DAG	1.3E+01	8.1E+02

pretraining-based method YaTC is relatively more robust due to its pretraining on large-scale unlabeled data.

The strong performance of BPF-DAG in few-shot settings can be attributed to the use of GNNs, which reinforce the interconnected patterns of network flows through message passing in the graph structure. This message-passing mechanism allows BPF-DAG to capture relationships between flows, making it more resilient to limited training data.

E. Complexity Analysis

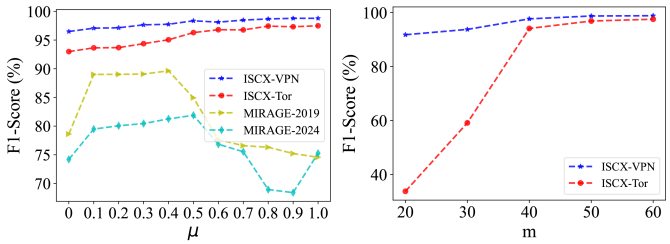
To thoroughly assess the trade-off between performance and complexity, we calculate the floating point operations (FLOPs) and model parameter size for both our method and the comparison methods. As shown in Table IX, our model' s FLOPs and parameter size are approximately 11% and 28% lower, respectively, than those of the SOTA methods.

According to Tables V and VI, it is concluded that BPF-DAG provides a significant performance improvement with tolerable model complexity. While BPF-DAG has higher computational complexity than traditional DL-based methods due to its use of GNN, it delivers superior accuracy. Conversely, although pre-training methods like ET-BERT produce similar classification results to BPF-DAG, their FLOPs and model parameters are several times higher than it. Besides, the pretraining process is time-consuming and excessively high computational complexity is not suitable for real-time traffic detection. To sum up, the computational complexity of BPF-DAG is slightly higher than some classical methods, but it achieves better classification performance compared to SOTA methods.

F. Parameter Sensitivity Analysis

In this part, we analyze how the classification performance is affected by the sensitivity of two crucial hyperparameters, i.e., the interpolation coefficient μ and raw byte length m.

1) The Impact of Interpolation Coefficient μ: The interpolation coefficient μ controls the tradeoff between the temporal representation and topological representation. $\mu = 0$ means only the temporal information of raw traffic is leveraged. When $0 < \mu < 1$, the temporal and topological representations are fully fused and μ controls the balance between them. The value of μ for optimal classification performance is different for different traffic datasets. Figure 10(a) shows the F1-score results with μ ranging from 0 to 1 for ISCX-VPN, ISCX-Tor, MIRAGE2019 and MIRAGE2024 datasets.



(a) Interpolation coefficient μ (b) Raw byte length m

Fig. 10. Parameter Sensitivity Analysis w.r.t. interpolation coefficient and raw byte length on ISCX-VPN and ISCX-Tor datasets.

As μ increasing from zero, the F1-score consistently increases, which exactly reflects the effectiveness of features fusion. The results show that the optimal values of μ vary across datasets and do not follow an obvious pattern. Specifically, the optimal values are 0.9 and 0.8 for the ISCX-VPN and ISCX-Tor datasets, respectively, and 0.4 and 0.5 for the MIRAGE-2019 and MIRAGE-2024 datasets, respectively. In conclusion, we can state that appropriately incorporating flow-level topological patterns into flow representations enables a significant improvement in classification performance.

2) The Impact of Raw Byte Length m: In our experiment, we truncate the first m bytes of each packet header to represent the raw packet. The value of m effects the tradeoff between computational overhead and classification performance. From Figure 10(b), we can find that the appropriate range of m is from 50 to 60 and the degree of influence varies for different datasets. The decrease of m causes obvious performance degradation for Tor dataset, while the VPN dataset is less affected. To balance the performance and overhead, m is set to 60 for all ISCX datasets.

VI. CONCLUSION

In this paper, we propose BPF-DAG, a byte-packet-flow features fusion-based reliable encrypted traffic classification method via dynamic attributed graph. This is the first method that integrates temporal packet relations into flow interaction patterns using raw-byte traffic as input, realizing byte-packet-flow multi-granularity features fusion. By jointly training a GNN and Transformer, BPF-DAG extracts topological patterns from the flow interaction graph and temporal characteristics from raw byte-based packet sequences. We propose a novel multi-granularity features fusion strategy to fully integrate byte-level, packet-level and flow-level traffic features. During training, the temporal representations of flows are learned from packet sequences using the optimized Transformer after each epoch, and then corresponding directed edge attributes are updated with them. To reduce the memory overhead, we leverage a technique called Memory Bank to track all temporal representations to decouple the training batch size from the total number of edges. Our method has been extensively evaluated on four encrypted traffic datasets, and the experimental results demonstrate that BPF-DAG not only outperforms SOTA methods in terms of accuracy, but also exhibits relatively lower complexity. Furthermore, BPF-DAG exhibits strong performance in few-shot learning scenarios. Looking

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on September 18,2026 at 06:22:17 UTC from IEEE Xplore. Restrictions apply.

ahead, we plan to extend our multi-granularity features fusion strategy to unsupervised Intrusion Detection Systems to detect and identify malicious traffic in real-time.

REFERENCES

[1] R. Sharma, S. Dangi, and P. Mishra, "A comprehensive review on encryption based open source cyber security tools," in *Proc. 6th Int. Conf. Signal Process., Comput. Control (ISPCC)*, Oct. 2021, pp. 614–619.

[2] E. Papadogiannaki and S. Ioannidis, "A survey on encrypted network traffic analysis applications, techniques, and countermeasures," *ACM Comput. Surv.*, vol. 54, no. 6, pp. 1–35, Jul. 2022.

[3] E. Ramadhani, "Anonymity communication VPN and tor: A comparative study," *J. Phys., Conf. Ser.*, vol. 983, Mar. 2018, Art. no. 012060.

[4] K. Thomas et al., "Data breaches, phishing, or malware? Understanding the risks of stolen credentials," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, Oct. 2017, pp. 1421–1434.

[5] G. Pellegrino, M. Johns, S. Koch, M. Backes, and C. Rossow, "Deemon: Detecting CSRF with dynamic analysis and property graphs," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, Oct. 2017, pp. 1757–1771.

[6] B. Anderson and D. McGrew, "Machine learning for encrypted malware traffic classification: Accounting for noisy labels and non-stationarity," in *Proc. 23rd ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining*, Halifax, NS, Canada, Aug. 2017, pp. 1723–1732.

[7] T. Van Ede et al., "FlowPrint: Semi-supervised mobile-app fingerprinting on encrypted network traffic," in *Proc. Netw. Distrib. Syst. Secur. Symp. (NDSS)*, vol. 27, Feb. 2020, pp. 1–18.

[8] Q. Zhou, L. Wang, H. Zhu, T. Lu, and V. S. Sheng, "WF-transformer: Learning temporal features for accurate anonymous traffic identification by using transformer networks," *IEEE Trans. Inf. Forensics Security*, vol. 19, pp. 30–43, 2024.

[9] Y. Mirsky, T. Doitshman, Y. Elovici, and A. Shabtai, "Kitsune: An ensemble of autoencoders for online network intrusion detection," 2018, *arXiv:1802.09089*.

[10] M. Roesch et al., "Snort: Lightweight intrusion detection for networks," *Lisa*, vol. 99, no. 1, pp. 229–238, 1999.

[11] V. Paxson, "Bro: A system for detecting network intruders in real-time," *Comput. Netw.*, vol. 31, nos. 23–24, pp. 2435–2463, Dec. 1999.

[12] V. F. Taylor, R. Spolaor, M. Conti, and I. Martinovic, "AppScanner: Automatic fingerprinting of smartphone apps from encrypted network traffic," in *Proc. IEEE Eur. Symp. Secur. Privacy*, Mar. 2016, pp. 439–454.

[13] A. Panchenko et al., "Website fingerprinting at internet scale," in *Proc. Netw. Distrib. Syst. Secur. Symp. (NDSS)*, vol. 1, 2016, p.23477.

[14] A. S. Jacobs, R. Beltiukov, W. Willinger, R. A. Ferreira, A. Gupta, and L. Z. Granville, "AI/ML for network security: The emperor has no clothes," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, Nov. 2022, pp. 1537–1551.

[15] J. Gu and S. Lu, "An effective intrusion detection approach using SVM with naïve Bayes feature embedding," *Comput. Secur.*, vol. 103, Apr. 2021, Art. no. 102158.

[16] G. Aceto, D. Ciuonzo, A. Montieri, V. Persico, and A. Pescapé, "AI-powered internet traffic classification: Past, present, and future," *IEEE Commun. Mag.*, vol. 62, no. 9, pp. 168–175, Sep. 2024.

[17] Z. Khodaverdian, H. Sadr, and S. A. Edalatpanah, "A shallow deep neural network for selection of migration candidate virtual machines to reduce energy consumption," in *Proc. 7th Int. Conf. Web Res. (ICWR)*, May 2021, pp. 191–196.

[18] H. Sadr, M. Nazari Soleimandarabi, and Z. Khodaverdian, "Automatic assessment of short answers based on computational and data mining approaches," *J. Decisions Oper. Res.*, vol. 6, no. 2, pp. 242–255, 2021.

[19] M. Nazari, H. Emami, R. Rabiei, A. Hosseini, and S. Rahmatizadeh, "Detection of cardiovascular diseases using data mining approaches: Application of an ensemble-based model," *Cognit. Comput.*, vol. 16, no. 5, pp. 2264–2278, Sep. 2024.

[20] H. Sadr, A. Salari, M. T. Ashoobi, and M. Nazari, "Cardiovascular disease diagnosis: A holistic approach using the integration of machine learning and deep learning models," *Eur. J. Med. Res.*, vol. 29, no. 1, p. 455, Sep. 2024.

[21] W. Wang, M. Zhu, J. Wang, X. Zeng, and Z. Yang, "End-to-end encrypted traffic classification with one-dimensional convolution neural networks," in *Proc. IEEE Int. Conf. Intell. Secur. Informat. (ISI)*, Jul. 2017, pp. 43–48.

[22] P. Sirinam, M. Imani, M. Juarez, and M. Wright, "Deep fingerprinting: Undermining website fingerprinting defenses with deep learning," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur. (CCS)*, Toronto, ON, Canada, Oct. 2018, pp. 1928–1943.

[23] C. Liu, L. He, G. Xiong, Z. Cao, and Z. Li, "FS-Net: A flow sequence network for encrypted traffic classification," in *Proc. IEEE INFOCOM Conf. Comput. Commun.*, Apr. 2019, pp. 1171–1179.

[24] M. Lopez-Martin, B. Carro, A. Sanchez-Esguevillas, and J. Lloret, "Network traffic classifier with convolutional and recurrent neural networks for Internet of Things," *IEEE Access*, vol. 5, pp. 18042–18050, 2017.

[25] K. Lin, X. Xu, and H. Gao, "TSCRNN: A novel classification scheme of encrypted traffic based on flow spatiotemporal features for efficient management of IIoT," *Comput. Netw.*, vol. 190, May 2021, Art. no. 107974.

[26] G. Bovenzi et al., "Benchmarking class incremental learning in deep learning traffic classification," *IEEE Trans. Netw. Service Manage.*, vol. 21, no. 1, pp. 51–69, Feb. 2024.

[27] A. Nascita, A. Montieri, G. Aceto, D. Ciuonzo, V. Persico, and A. Pescapé, "Improving performance, reliability, and feasibility in multimodal multitask traffic classification with XAI," *IEEE Trans. Netw. Service Manage.*, vol. 20, no. 2, pp. 1267–1289, Jun. 2023.

[28] T. N. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," 2016, *arXiv:1609.02907*.

[29] Y. Chen and Y. Wang, "MPAF: Encrypted traffic classification with multi-phase attribute fingerprint," *IEEE Trans. Inf. Forensics Security*, vol. 19, pp. 7091–7105, 2024.

[30] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Lió, and Y. Bengio, "Graph attention networks," 2017, *arXiv:1710.10903*.

[31] W. L. Hamilton, R. Ying, and J. Leskovec, "Inductive representation learning on large graphs," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 30, 2017, pp. 1024–1034.

[32] H. Shi, H. Li, D. Zhang, C. Cheng, and X. Cao, "An efficient feature generation approach based on deep learning and feature selection techniques for traffic classification," *Comput. Netw.*, vol. 132, pp. 81–98, Feb. 2018.

[33] R. Zhao et al., "Yet another traffic classifier: A masked autoencoder based traffic transformer with multi-level flow representation," in *Proc. AAAI Conf. Artif. Intell.*, 2023, vol. 37, no. 4, pp. 5420–5427.

[34] X. Lin, G. Xiong, G. Gou, Z. Li, J. Shi, and J. Yu, "ET-BERT: A contextualized datagram representation with pre-training transformers for encrypted traffic classification," in *Proc. ACM Web Conf.*, Apr. 2022, pp. 633–642.

[35] X. Hu, W. Gao, G. Cheng, R. Li, Y. Zhou, and H. Wu, "Toward early and accurate network intrusion detection using graph embedding," *IEEE Trans. Inf. Forensics Security*, vol. 18, pp. 5817–5831, 2023.

[36] W. W. Lo, S. Layeghy, M. Sarhan, M. Gallagher, and M. Portmann, "E-GraphSAGE: A graph neural network based intrusion detection system for IoT," in *Proc. IEEE/IFIP Netw. Oper. Manage. Symp.*, Budapest, Hungary, Apr. 2022, pp. 1–9.

[37] G. Duan, H. Lv, H. Wang, and G. Feng, "Application of a dynamic line graph neural network for intrusion detection with semisupervised learning," *IEEE Trans. Inf. Forensics Security*, vol. 18, pp. 699–714, 2023.

[38] H. Zhang et al., "TFE-GNN: A temporal fusion encoder using graph neural networks for fine-grained encrypted traffic classification," in *Proc. ACM Web Conf.*, Apr. 2023, pp. 2066–2075.

[39] M. Wang, N. Yang, and N. Weng, "K-GetNID: Knowledge-guided graphs for early and transferable network intrusion detection," *IEEE Trans. Inf. Forensics Security*, vol. 19, pp. 7147–7160, 2024.

[40] I. Akbari et al., "A look behind the curtain: Traffic classification in an increasingly encrypted web," *Proc. ACM Meas. Anal. Comput. Syst.*, vol. 5, no. 1, pp. 1–26, 2021.

[41] B. Anderson, S. Paul, and D. McGrew, "Deciphering malware's use of TLS (without decryption)," *J. Comput. Virol. Hacking Techn.*, vol. 14, no. 3, pp. 195–211, Aug. 2018.

[42] W. Wang, M. Zhu, X. Zeng, X. Ye, and Y. Sheng, "Malware traffic classification using convolutional neural network for representation learning," in *Proc. Int. Conf. Inf. Netw. (ICOIN)*, Da Nang, Vietnam, Jan. 2017, pp. 712–717.

[43] W. Wang et al., "HAST-IDS: Learning hierarchical spatial–temporal features using deep neural networks to improve intrusion detection," *IEEE Access*, vol. 6, pp. 1792–1806, 2018.

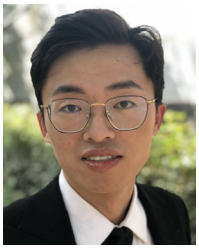
[44] A. Nascita, A. Montieri, G. Aceto, D. Ciuonzo, V. Persico, and A. Pescapé, "XAI meets mobile traffic classification: Understanding and improving multimodal deep learning architectures," *IEEE Trans. Netw. Service Manage.*, vol. 18, no. 4, pp. 4225–4246, Dec. 2021.

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on September 18,2026 at 06:22:17 UTC from IEEE Xplore. Restrictions apply.

[45] G. Zhou, X. Guo, Z. Liu, T. Li, Q. Li, and K. Xu, "TrafficFormer: An efficient pre-trained model for traffic data," in *Proc. IEEE Symp. Secur. Privacy (SP)*, May 2025, pp. 1844–1860.
[46] P. Lin, K. Ye, Y. Hu, Y. Lin, and C.-Z. Xu, "A novel multimodal deep learning framework for encrypted traffic classification," *IEEE/ACM Trans. Netw.*, vol. 31, no. 3, pp. 1369–1384, Jun. 2023.
[47] A. Vaswani et al., "Attention is all you need," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 30, 2025, pp. 5998–6008.
[48] A. Sriram, H. Jun, S. Satheesh, and A. Coates, "Cold fusion: Training Seq2Seq models together with language models," 2017, *arXiv:1708.06426*.
[49] Z. Wu, Y. Xiong, S. X. Yu, and D. Lin, "Unsupervised feature learning via non-parametric instance discrimination," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, Jun. 2018, pp. 3733–3742.
[50] K. He, H. Fan, Y. Wu, S. Xie, and R. Girshick, "Momentum contrast for unsupervised visual representation learning," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2020, pp. 9729–9738.
[51] G. Draper-Gil, A. H. Lashkari, M. S. I. Mamun, and A. A. Ghorbani, "Characterization of encrypted and VPN traffic using time-related," in *Proc. 2nd Int. Conf. Inf. Syst. Security Privacy (ICISSP)*, 2016, pp. 407–414.
[52] A. H. Lashkari, G. D. Gil, M. S. I. Mamun, and A. A. Ghorbani, "Characterization of Tor traffic using time based features," in *Proc. 3rd Int. Conf. Inf. Syst. Secur. Privacy (ICISSP)*, Feb. 2017, pp. 253–262.
[53] G. Aceto, D. Ciuonzo, A. Montieri, V. Persico, and A. Pescapé, "MIRAGE: Mobile-app traffic capture and ground-truth creation," in *Proc. 4th Int. Conf. Comput., Commun. Secur. (ICCCS)*, Oct. 2019, pp. 1–8.
[54] I. Guarino, D. Ciuonzo, A. Montieri, and A. Pescapè, "Mirage-AppxAct-2024: A novel dataset for mobile app and activity traffic analysis," in *Proc. 20th Int. Conf. Wireless Mobile Comput., Netw. Commun. (WiMob)*, Oct. 2024, pp. 663–666.



Yunxiao Shi (Student Member, IEEE) received the B.S. degree in communication engineering from Nanjing University of Posts and Telecommunications, Nanjing, China, in 2023. He is currently pursuing the Ph.D. degree with the School of Electronic Information and Electrical Engineering, Shanghai Jiao Tong University, Shanghai, China. His research interests include encrypted traffic classification, intrusion detection systems, and machine learning for cyber security.



Gaolei Li (Member, IEEE) is currently an Associate Professor with the School of Electronic Information and Electrical Engineering, Shanghai Jiao Tong University, Shanghai, China. He has published more than 100 papers, including IEEE TRANSACTIONS ON INFORMATION FORENSICS AND SECURITY (IEEE TIFS), IEEE TRANSACTIONS ON DEPENDABLE AND SECURE COMPUTING (IEEE TDSC), NDSS, ACM CCS, ACM MM, and AAAI. His research interests include machine learning security and privacy-preserving. He has obtained many awards, including the Outstanding Paper Award of IEEE ISPA 2024, the Third Prize of Science and Technology Progress Award of China Electric Power Development Acceleration Committee in 2024, the Best Conference Paper Award of China Cryptology Society in 2020, and the Best Conference Paper Award of IEEE CSIM Committee in 2018. He also served as a TPC member for CVPR 2024–2025, AAAI 2023&2024&2025, ACM MM 2023&2024&2025, and ICLR 2025. Besides, he also serves a reviewer for IEEE TIFS, IEEE TDSC, IEEE TRANSACTIONS ON MOBILE COMPUTING, IEEE JOURNAL ON SELECTED AREAS IN COMMUNICATIONS, IEEE/ACM TRANSACTIONS ON NETWORKING, IEEE TRANSACTIONS ON SERVICES COMPUTING, IEEE TRANSACTIONS ON NEURAL NETWORKS AND LEARNING SYSTEMS, and IEEE TRANSACTIONS ON GREEN COMMUNICATIONS AND NETWORKING.



Jun Wu (Senior Member, IEEE) received the Ph.D. degree in information and telecommunication studies from Waseda University, Japan, in 2011. He was a Post-Doctoral Researcher with the Research Institute for Secure Systems, National Institute of Advanced Industrial Science and Technology (AIST), Japan, from 2011 to 2012. He was a Researcher with the Global Information and Telecommunication Institute, Waseda University, from 2011 to 2013. He is currently a Professor with the School of Electronic Information and Electrical Engineering, Shanghai Jiao Tong University, China. He is also the Dean of the Institute of Cyber Science and Technology and the Vice Director of the National Engineering Research Center for Information Content Analysis Technology, Shanghai Jiao Tong University. He has hosted and participated in a lot of research projects, including the National Natural Science Foundation of China (NFSC), the National 863 Plan and 973 Plan of China, and Japan Society of the Promotion of Science Projects (JSPS). His research interests include the intelligence and security techniques of artificial intelligence, the Internet of Things (IoT), 5G/6G, and molecular communication. He has been the Track Chair of VTC 2019 and VTC 2020 and a TPC Member of more than ten international conferences, including ICC and GLOBECOM. He is the Chair of the IEEE P21451-1-5 Standard Working Group. He has been a Guest Editor of IEEE TRANSACTIONS ON INDUSTRIAL INFORMATICS, IEEE TRANSACTIONS ON INTELLIGENT TRANSPORTATION SYSTEMS, IEEE SENSORS JOURNAL, and *Sensors*. He is an Associate Editor of the IEEE SYSTEMS JOURNAL and IEEE NETWORKING LETTERS.



Jianhua Li (Senior Member, IEEE) is currently a Professor/Ph.D. Supervisor and the Dean of the Institute of Cyber Science and Technology, Shanghai Jiao Tong University, Shanghai, China. He is also the Director of the National Engineering Laboratory for Information Content Analysis Technology and the Engineering Research Center for Network Information Security Management and Service of Chinese Ministry of Education. He has published more than 400 papers, including IEEE TRANSACTIONS ON DEPENDABLE AND SECURE COMPUTING, IEEE TRANSACTIONS ON INFORMATION FORENSICS AND SECURITY, IEEE TRANSACTIONS ON MOBILE COMPUTING, IEEE TRANSACTIONS ON EMERGING TOPICS IN COMPUTATIONAL INTELLIGENCE, ACM CCS, ACM MobiHoc, IEEE Infocom, and IEEE Globecom. He has published six books and has about 20 patents. He made three standards and has five software copyrights. His research interests include information security, signal processing, and computer network communication. He has got many awards, including the Second Prize of National Technology Progress Award of China in 2005, the First Prize of Science and Technology Progress Award of the Ministry of Education, the First Prize of Science and Technology Progress Award of Shanghai, the IEEE CSIM Committee Best Paper Award in 2016, the IEEE TETC Best Paper Award in 2017, and the ESI Highly Cited Scientist from 2022 to 2024.



He Fang (Member, IEEE) received the Ph.D. degree in electrical and computer engineering from Western University, Canada, in 2020. She is currently a Full Professor with Fujian Normal University, China. Her research interests include intelligent security provision and trust management. She has over 60 peer-reviewed journals and conference papers, and won the Best Paper Award from IEEE GLOBECOM 2023. She serves as an Associate Editor for several journals, including IEEE TRANSACTIONS ON INFORMATION FORENSICS AND SECURITY and *China Communications*. She was also involved in many IEEE conferences as the track chair or the session chair.

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on September 18,2026 at 06:22:17 UTC from IEEE Xplore. Restrictions apply.